Summary of Agentic AI Systems 3/24

Dillan Morell (djmorell), Téa Hajratwala (hajratwt), Nidhil Nayudu (nnayudu)

Parrot: Efficient Serving of LLM-based Applications with Semantic Variable

Problem and Motivation

The rise of LLMs has enabled LLM-based applications like AI agents or co-pilots. Many of these applications design complex workflows using multiple LLM requests to accomplish a single task like map-reduce summarizations, thought chains, and multi-agent coding processes. However, public LLM services do not consider the context behind these workflows. Rather, they use an over-simplified request-level API that considers each request individually. Therefore, they lack any knowledge on data dependencies between requests or the structure of the overall prompts, leading to solely intra-request optimizations and no inter-request optimizations. That creates several issues with serving LLM-based applications that leads to suboptimal end-to-end performance.

Excessive Overhead of Consecutive Requests: With the current LLM service infrastructure, developers have to make a request, parse the output, compose a prompt for the next request, and continue repeating this process until they finish their workflow. This leads to significant network latency since about 30 ~ 50% of end-to-end latency originates outside the LLM engine. Also, this sequential method of requesting leads to queueing delays since other requests will likely arrive between any two consecutive requests that any given developer makes.

Misaligned Scheduling Objectives: In complex workflows that chain together several LLM calls, developers care more about end-to-end latency rather than per-request latency, which LLMs currently optimize for. To minimize end-to-end latency, providers need to trade off per-request latency for throughput by processing higher batch sizes for parallel tasks to reach the next sequential task as quickly as possible. A common example of this is in a MapReduce workflow, where running as many map tasks as possible in parallel leads to the reduce stage starting sooner, and thus, reducing end-to-end latency.

Redundant Computations: Most LLM-based applications have a lot of commonality across prompts. For example, many of them have the same system prompts for instructions on how to respond or safety checks, leading to more than 94% of prefix tokens that could be repetitively used across requests for various users. Since LLMs treat every request independently, they repeatedly load and compute the KV cache with the same values which wastes significant amounts of GPU compute.

Related Works

Prior research related to Parrot can be grouped into deep learning serving systems, LLM orchestrator frameworks, and DAG-aware system optimizations.

Deep learning serving systems explored aspects of model deployment like batching, caching, placement, scheduling, and model parallelism for serving models. These systems like AlpaServe were designed for more general deep learning models and don't align exactly with the requirements of LLMs such as autoregressive decoding. Systems like Orca and vLLM proposed techniques like continuous batching and PagedAttention, respectively. These systems are great at improving request throughput and memory utilization. However, they still consider each request separately. So, all of the problems previously described still exist.

LLM orchestrator frameworks attempt to help developers design more complex workflows by representing workflow patterns and semantic functions as a graph in frameworks like LangChain and PromptFlow. But all of this only exists on the client-side, and ends up being executed sequentially on the LLM server, so the same issues are still prevalent.

DAG-aware system optimizations have been proposed for many other applications. For example, Tez and Dryad use task dependencies to optimize scheduling data analytic workloads. Parrot aims to draw inspiration from these existing works and apply similar optimizations to serving LLMs. This way, they can share context and optimize the KV cache in addition to solving the existing problems on the server-side.

Solution Overview

Parrot is a system that optimizes end-to-end latency of workflows in LLM-based applications by exposing application-level knowledge to the LLM service system through Semantic Functions and Semantic Variables. Instead of processing each request separately, Parrot treats each LLM call as a Semantic Function which is a specific task defined by an LLM prompt. Within the prompt of a Semantic Function, developers define Semantic Variables which act as placeholders for the inputs and outputs. So for example, say there is a software engineer agent and a QA engineer agent in a multi-agent workflow. The software engineer agent's Semantic Function would generate a code output variable, which is then mapped to be the input Semantic Variable for the QA engineer's Semantic Function. By creating this data pipeline, Parrot preserves the prompt structure for inter-request analysis on the server-side.

Unlike a traditional completion API, Parrot splits the request into a submit and a get operation. Calling the Semantic Function triggers the submit API to submit a LLM request with its prompt and input Semantic Variables. And once the get API is explicitly called, it fetches the values of the output Semantic Variable from the LLM service. By calling the submit API upfront, the server can analyze how the variables are connected and build a directed acyclic graph (DAG) of the workflow. Parrot uses a graph-based executor for this DAG to serve dependent requests, so that once all producer requests are finished, it can add them directly to the internal message queue. This removes all of the typical network latency which also allows the message to enter the queue earlier.

Architecturally, Parrot has three layers. The API layer interfaces with the developer front end to receive the submitted Semantic Functions and Variables. The Parrot Manager then acts as a control plane. It takes those submitted requests, constructs the DAG, manages inter-request dependencies, and adjusts the batch sizes to optimize throughput for parallel tasks to minimize end-to-end latency. The requests end up getting routed to the Parrot LLM Engine which actually handles the context management and GPU execution.

To address the redundant computation problem that the authors highlighted, they used a PrefixHash to identify requests that share the same system prompts. The scheduler can quickly check this to co-locate these requests on the same GPU. This way, the KVCache is only loaded into shared memory once and the computed attention metrics can be reused for multiple requests. This accelerates complex workflows since many of the steps tend to have the same system prompt.

By eliminating iterative requests between the client and server and KV cache usage, Parrot reduces non-inference latency. For complex workflows, Parrot achieves end-to-end latency speedups of up to 11.7x when compared against their baseline which was vLLM. So they have shown that exposing application-level knowledge to the LLM service produces considerable speedups.

Limitations

First, Parrot has limited support for dynamic workflows (such as conditionals). Second, to keep things secure, Parrot intentionally refuses to run custom ‘native’ code (like a Python script) on its own servers; however, this makes things slower in this case because of network latency. Third, Parrot assumes workflows can be represented with a DAG, when in reality there are some applications (namely dynamic ones) which aren’t compatible with this representation.

Future Research Directions

Future research directions for Parrot include the introduction of dynamic control flow, which would allow the system to handle native functions and conditional connections that currently require client-side execution. This advancement would support speculative execution by allowing Parrot to pre-compute high-probability application branches based on profiling information. Furthermore, the system's inter-request analysis could be extended to tackle complex cluster-level issues such as fair scheduling, failure handling, and the optimization of heterogeneous clusters. Finally, the researchers seek to improve integration with popular orchestration frameworks like LangChain, SemanticKernel, and PromptFlow to ensure important application-level information is exposed to the LLM service.

Summary of Class Discussion

Q: Savings come from reusing shared prompts. If one of these is evicted, the savings go away, so what caching policy is the best to use?
A: The paper doesn’t mention this, but we assume using basic KVCache policies like LRU eviction. Pre context method in cache allows developers to explicitly program entries into the KVCache (they can write their own algorithm).

Q: What if a user is using semantic variables which change a lot (ie. timestamp), will that mess up prefix sharing?
A: No it will not. The prompt goes into the store without rendering. Even if the semantic variable is changing, the prompt structure is the same so it won’t mess up.

Q: In the case of models supporting super long contexts, does that take away things like chunking?
A: If the prompt window can be expanded, you don't need as many chunks, and you don't need as many requests that you have to send.

Q: Would we want to separate between agentic vs non agentic workloads?
A: In the case of long documents, the current system works well. For parallel task groups, parrot would work well. For very generalized questions with no interdependence, there would probably be added overhead. PIE may have more advantages since you can customize based on what you want to optimize by.

Q: Could a nefarious developer mark all their requests as latency forward and therefore hog the GPU?
A: If both requests are for latency, they will be grouped together on the same LLM engine which is designed for latency. Parrot will take the most constrained approach to maximize throughput. Plus, work mainly happens on the client side. This is a typical problem among all resource management problems. The common solution is to create a punishment if developers lie about priority. In other words, incentivize telling the truth.

Q: Why does Parrot use prefix hashing at all? Why not just use the semantic variables?
A: If we only match on semantic var, we lose all context from the rest of the prompt.

Q: Without dependencies, would Parrot be worth it?

A: No. Its main function is to organize dependencies.

Pie: A Programmable Serving System for Emerging LLM Applications

Problem and Motivation

Their key assumption is that modern LLM applications have outgrown the design assumptions of current serving systems. Existing systems were built around a monolithic prefill-and-decode loop optimized for standard text completion. That design works well when each request is just a prompt followed by sequential token generation, but it becomes restrictive for newer applications involving things like reasoning trees, speculative decoding, external tools, agents, and multimodal or interactive workflows.

They argue there are three core limitations in current serving systems.

Implicit KV cache management: KV cache management is too implicit and system-wide, relying on policies like LRU or prefix sharing rather than letting the application decide what to do with each part of the cache. This makes advanced methods like Graph-of-Thought, Recursion-of-Thought, beam search, and attention sink difficult to support cleanly.

Inflexible decoding process: Decoding is too inflexible because the token prediction and sampling loop is tightly coupled, making it hard to implement per-request or per-step methods such as speculative decoding, constrained decoding, watermarking, or MCTS-style generation.

Poor workflow integration: Current systems integrate poorly with external computation and I/O. Agentic workflows often require tool calls, API requests, symbolic checks, or code execution, but today this usually forces control back to the client, causing extra network latency and repeated prefills.

The motivation for Pie is therefore to redesign LLM serving around programmability. Instead of making every new optimization a system-level patch, the paper wants application developers to use custom serving logic directly while still retaining adequate performance.

Related Works

In LLM serving, Pie compares itself most directly to systems such as vLLM, TGI, SGLang, and Parrot. SGLang and Parrot already introduce some programmatic abstractions, such as fork/join style control and semantic variables, but the authors argue these systems still retain a fundamentally monolithic generation process. As a result, they do not expose fine-grained control over low-level resources, inference steps, or integrated I/O to the extent Pie does.

The paper also connects to programming literature, including LMQL, Guidance, Outlines, XGrammar, and AICI. These systems mainly focus on controlling generation semantics, often through grammars or declarative constraints. Pie differs by allowing a Turing-complete program to control what is generated and how generation is executed during each step.

Finally, Pie is positioned as complementary to frameworks such as LangChain, LangFlow, DSPy, and AutoGen. Those tools help developers build workflows and prompts, while Pie can serve as a backend

that executes such workflows more efficiently. The paper also notes that Pie builds on a broader systems tradition of programmable infrastructures in operating systems, networking, and distributed systems.

Solution Overview

Pie's main idea is to break the traditional monolithic token generation loop into fine-grained handlers and hand control to user-defined programs called inferlets. Instead of the serving system deciding everything about embedding, forward passes, sampling, cache usage, and I/O, inferlets explicitly orchestrate these pieces through APIs.

Model: The programming model revolves around three forward-pass stages: embed, forward, and sample. Inferlets use APIs for each stage to define custom generation workflows. Pie also introduces explicit resources such as Embed objects for token embeddings and KvPage objects for chunks of KV cache. These are allocated, manipulated, shared, and deallocated directly by inferlets, giving the application precise control over memory and cache behavior. Command queues make dependencies between operations explicit and help the scheduler batch compatible API calls efficiently.

Architecture: Architecturally, Pie has three layers. The application layer runs inferlets in a WebAssembly runtime, which provides lightweight sandboxing and portability. The control layer manages inferlet lifecycles, virtualized resources, scheduling, and API batching. The inference layer performs the GPU-side execution of batched operations. This separation lets Pie preserve flexibility at the application level while still exploiting batching and hardware acceleration below.

Results: The evaluation claims that Pie matches state-of-the-art performance on standard text completion with only modest overhead, around 3–12% latency overhead depending on model size, while outperforming baseline systems substantially on more advanced workflows. For agentic and reasoning-heavy tasks, Pie achieves lower latency and higher throughput because inferlets can keep KV cache state across steps, avoid repeated client-server round trips, and apply application-specific optimizations.

Limitations

First, Pie has a lot of programming complexity since it has 42 fine-grained APIs that require developers to manually orchestrate workflows, a task the authors compare to low-level OpenGL programming. Second, unlike many existing systems that use implicit heuristics, Pie demands that users explicitly manage memory resources such as KV cache pages and embeddings through manual allocation and deallocation calls. Third, the system relies on a single centralized control layer and batch scheduler to coordinate interactions, which may create bottlenecks. Fourth, Pie's current evaluation is relatively narrow, having been tested primarily on single NVIDIA L4 GPUs using the Llama model family. Finally, the system is not distributed in its current form, as it lacks the cross-node coordination, globally-aware scheduling, and fault tolerance required for large scale production deployments.

Future Research Directions

Because of these limitations, future work for Pie includes (1) distributing the system to make the system more scalable, (2) providing a higher level of abstraction to make the developer experience more user friendly, and (3) improving the overall security of Pie. For the final point, the authors suggest incorporating "capability-based I/O, per-inferlet rate limits and quotas, and limiting or obfuscating exposure of logits (e.g., top-*K* caps)."

Summary of Class Discussion

Q: What sort of workloads/cases best leverage Pie's structure?
A: Big parameter models are best for Pie. Specifically, for an 8B parameter model, the overhead is only 1.53 ms, or 2.39% of the total time per output token. (In contrast, a smaller 1B model sees a much higher overhead of 11.41%)

Q: Would we want to separate between agentic vs non agentic workloads?
A: For long documents, the current system works well. For parallel task groups, parrot would work well. For very generalized questions with no interdependence, there would probably be added overhead. Pie may have more advantages since you can customize based on what you want to optimize by.

Q: What about Pie's security concerns? Is it safe for hyperscale?
A: It's definitely not safe for large scale. The authors chose webassembly because it was lightweight. Pie is not scalable at all for larger systems, i.e. something like AWS. On top of this, Pie is not distributed, which adds complexity to scaling.

Q: If you know you have a trusted source for writing inferlets, is it worth using webassembly or can we bypass with a native language?
A: The authors chose Wasm because "native OS processes provide weaker containment" and other options like Docker add too much startup overhead. However, it was also noted that their own native C++/CUDA implementation of the inference layer (not the inferlets) achieved 10–30% lower latency than the Python version, suggesting that if a source is fully trusted and isolation isn't a priority, native code is faster.

Q: Can you use Pie maliciously/ not safely?
A: If the inferlets are not trusted, security can be compromised (DoS, misuse of access to token distributions).